

# POC Vault

Gestion des secrets & interaction via CLI et API REST

| Champ         | Valeur                          |
|---------------|---------------------------------|
| Projet        | POC Vault - Gestion des secrets |
| Auteur(s)     | Salah-Wassim ARFA               |
| Établissement | Epitech                         |
| Date          | 10/04/2026                      |
| Version       | 1.0                             |

## 1. Introduction

---

Ce document présente le POC (Proof of Concept) réalisé autour de HashiCorp Vault dans le cadre du projet CIA. Vault est un outil open-source de gestion des secrets, conçu pour stocker, contrôler l'accès et distribuer des informations sensibles telles que des mots de passe, des tokens ou des clés API.

### 1.1 Contexte

Dans le cadre du projet CIA, Vault est déployé en tant que solution centralisée de gestion des secrets pour l'infrastructure hybride multi-sites. Il constitue la brique de sécurité permettant de ne plus stocker de secrets en clair dans les fichiers de configuration ou les scripts.

### 1.2 Objectif du POC

Ce POC a pour objectif de déployer Vault en mode développement, de prendre en main son cycle de vie via un script Bash (démarrage / arrêt), puis de démontrer la création et la lecture de secrets en utilisant à la fois la CLI Vault et l'API REST.

## 2. Présentation de Vault

---

Vault fonctionne sur le principe d'un serveur de secrets sécurisé. Il organise les secrets dans des moteurs de secrets (secrets engines). Dans ce POC, nous utilisons le moteur KV (Key/Value), le plus simple, qui permet de stocker des paires clé/valeur de manière sécurisée.

| Composant                 | Détail                                       |
|---------------------------|----------------------------------------------|
| Version Vault             | v1.21.4                                      |
| Mode de démarrage         | Dev mode (vault server -dev)                 |
| Moteur de secrets utilisé | KV v2 (Key/Value)                            |
| Port d'accès              | HTTP sur le port 8200                        |
| Authentification          | Root token (généré au démarrage en dev mode) |
| Interfaces disponibles    | CLI (vault) et API REST (curl)               |

## 3. Installation & Déploiement

---

### 3.1 Prérequis

- Système Linux (Ubuntu Server Minimal)
- Binaire Vault installé sur la VM (téléchargé depuis [releases.hashicorp.com](https://releases.hashicorp.com))
- Variable d'environnement VAULT\_ADDR configurée
- Variable d'environnement VAULT\_TOKEN configurée avec le root token

### 3.2 Installation du binaire Vault

Vault s'installe sous la forme d'un unique binaire téléchargé depuis le site officiel de HashiCorp :

```
# Téléchargement et installation du binaire
wget https://releases.hashicorp.com/vault/1.21.4/vault_1.21.4_linux_amd64.zip
unzip vault_1.21.4_linux_amd64.zip
sudo mv vault /usr/local/bin/

# Vérification de l'installation
vault version
```

### 3.3 Script de gestion du cycle de vie

Un script Bash a été développé pour démarrer et arrêter le serveur Vault en mode développement. Ce mode est adapté au POC : il démarre Vault en mémoire, sans persistance des données, avec le chiffrement activé et un root token prédéfini.

❑ *Le dev mode ne doit pas être utilisé en production : les secrets sont stockés en mémoire et perdus à l'arrêt du serveur.*

```
#!/bin/bash

echo "====Démarrage de Vault======"

echo "Arrêt des instances existantes..."

killall -f "vault server" 2>/dev/null

sleep 2

echo "Nettoyage des données"
rm -f vault.log 2>/dev/null

sleep 1

vault server -dev \
    -dev-listen-address="0.0.0.0:8200" \
    -dev-root-token-id="rootToken" \
    &> vault.log &

echo "Attente du démarrage..."
sleep 5

export VAULT_PID=$!

export VAULT_ADDR="http://localhost:8200"
export VAULT_TOKEN="rootToken"

if vault status > /dev/null 2>&1; then
    echo "Connexion réussie"
else
    echo "Connexion échouée"
fi

echo -e ""
echo -e "Vault démarrer sur : $VAULT_ADDR"
echo -e "Token root : $VAULT_TOKEN"
echo -e "PID : $VAULT_PID"

echo ""

read -p "Voulez-vous créer un secret ? (y/n) " answer
```

```
case "$answer" in
  y|Y)
    read -p "Insérer un nom de chemin : " pathName
    read -p "Insérer le nom de votre clé : " keyName
    read -p "Insérer le secret que vous souhaitez créer : " secretkey
    vault kv put -mount=secret /$pathName $keyName=$secretkey;;
  n|N) return 0 ;;
  *) echo "Réponse incorrecte !"
esac
```

## 4. Gestion des secrets

### 4.1 Principe du moteur KV

Le moteur KV (Key/Value) de Vault permet de stocker des secrets sous forme de paires clé/valeur, organisées dans des chemins (paths). En version KV v2, chaque secret est versionné, ce qui permet de conserver l'historique des modifications.

| Opération                       | CLI Vault                    | API REST (méthode HTTP) |
|---------------------------------|------------------------------|-------------------------|
| Créer / mettre à jour un secret | <code>vault kv put</code>    | PUT / POST              |
| Lire un secret                  | <code>vault kv get</code>    | GET                     |
| Lister les secrets              | <code>vault kv list</code>   | LIST                    |
| Supprimer un secret             | <code>vault kv delete</code> | DELETE                  |

### 4.2 Via la CLI Vault

Configurer les variables d'environnement :

```
export VAULT_ADDR="http://localhost:8200"
export VAULT_TOKEN="rootToken"
```

Créer un secret :

```
vault kv put -mount=secret test/ hello=world
```

Lire un secret :

```
vault kv get secret/test
```

Lire uniquement la valeur d'un champ :

```
vault kv get -field=hello secret/test
```

Supprimer un secret :

```
vault kv delete secret/test
```

### 4.3 Via l'API REST

Vault expose également une API REST complète, ce qui permet d'interagir avec les secrets depuis n'importe quel script ou outil compatible HTTP.

Créer un secret :

```
curl \
-H "X-Vault-Token: rootToken" \
-H "Content-Type: application/json" \
-X POST \
-d '{"data": {"value": "monSuperPassword"}}' \
http://192.168.100.100:8200/v1/secret/data/test2
```

Lire un secret :

```
curl -X GET http://127.0.0.1:8200/v1/secret/data/test2 \
-H "X-Vault-Token: rootToken" | jq '.data.data'
```

Supprimer un secret :

```
curl -X DELETE http://127.0.0.1:8200/v1/secret/data/test2 \
-H "X-Vault-Token: rootToken"
```

## 5. Résultats obtenus

Le tableau ci-dessous synthétise l'ensemble des opérations testées lors du POC :

| Opération               | Via CLI      | Via API REST | Observations                                    |
|-------------------------|--------------|--------------|-------------------------------------------------|
| Démarrage de Vault      | ✓Fonctionnel | —            | Script Bash, mode dev, root token = "rootToken" |
| Arrêt de Vault          | ✓Fonctionnel | —            | Arrêt propre via le PID sauvegardé              |
| Création d'un secret    | ✓Fonctionnel | ✓Fonctionnel | Stockage dans secret/projet-cia                 |
| Lecture d'un secret     | ✓Fonctionnel | ✓Fonctionnel | Retour JSON parsé avec jq (API)                 |
| Suppression d'un secret | ✓Fonctionnel | ✓Fonctionnel | Réponse HTTP 204 (API)                          |

```

epicloud@vm2001:~/vault$ source ./manage-vault.sh
===Démarrage de Vault
Arrêt des instances existantes...
Nettoyage des données
Attente du démarrage...
Connexion réussie

Vault démarré sur : http://localhost:8200
Token root : rootToken
PID : 72869

```

Figure 1 Démarrage de Vault

```

epicloud@vm2001:~/vault$ vault kv put -mount=secret /test3 git=#monSuperMotDePasseGit$
== Secret Path ==
secret/data/test3

===== Metadata =====
Key          Value
---          -
created_time  2026-03-19T19:28:09.568914493Z
custom_metadata <nil>
deletion_time n/a
destroyed     false
version       1
epicloud@vm2001:~/vault$ |

```

Figure 2 Création d'un secret via CLI

```

epicloud@vm1001:~$ curl \
-H "X-Vault-Token: rootToken" \
-H "Content-Type: application/json" \
-X POST \
-d '{"data":{"value":"monSuperPassword"}}' \
http://192.168.100.100:8200/v1/secret/data/test2
{"request_id":"94645eb0-d27a-cdf2-4c65-c5f56eeal83f","lease_id":"","renewable":false,"lease_duration":0,"data":{"created_time":"2026-03-19T19:14:13.711889215Z","custom_metadata":null,"deletion_time":"","destroyed":false,"version":1},"wrap_info":null,"warnings":null,"auth":null,"mount_type":"kv"}

```

Figure 3 Création d'un secret via API REST depuis une VM distante

```

epicloud@vm2001:~/vault$ vault kv get -field=hello secret/test
world

```

Figure 4 Lecture d'un secret via CLI

```

epicloud@vm1001:~$ curl -X GET http://192.168.100.100:8200/v1/secret/data/test2 -H "X-Vault-Token: rootToken" | jq '.data.data'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
   100    346   100    346    0     0   197k      0  --:--:-- --:--:-- --:--:--   337k
{
  "value": "monSuperPassword"
}

```

Figure 5 Lecture d'un secret via API REST depuis une VM distante

```

epicloud@vm2001:~/vault$ vault kv delete secret/test
Success! Data deleted (if it existed) at: secret/data/test

```

Figure 6 Suppression d'un secret via la CLI

```
epicloud@vm1001:~$ curl \
> -H "X-Vault-Token: rootToken" \
> -X DELETE \
> http://192.168.100.100:8200/v1/secret/data/test2
epicloud@vm1001:~$ |
```

Figure 7 Suppression d'un secret via l'API REST depuis une VM distante

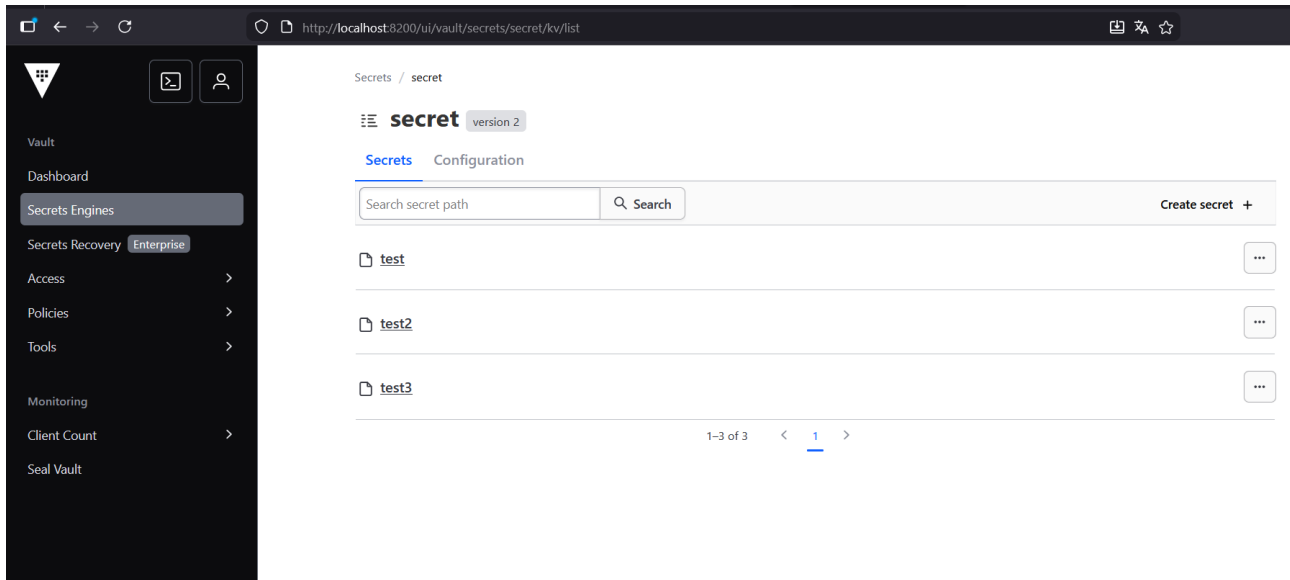


Figure 8 Interface GUI des secrets

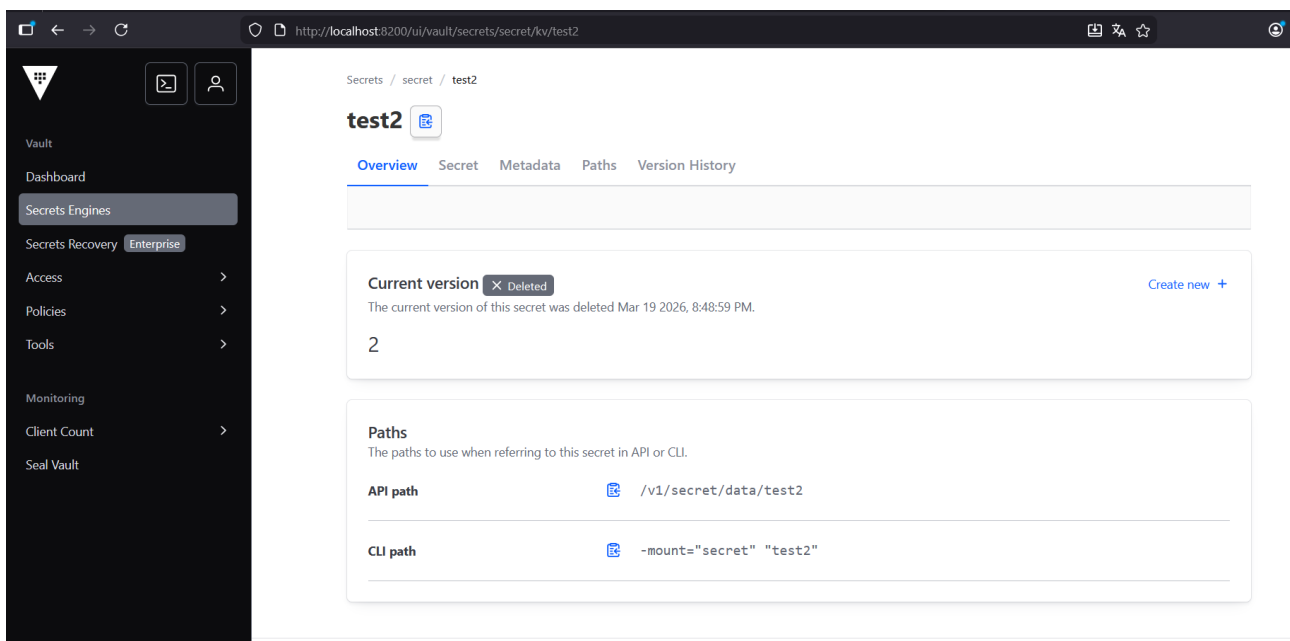


Figure 9 Interface GUI : secret test2 supprimer

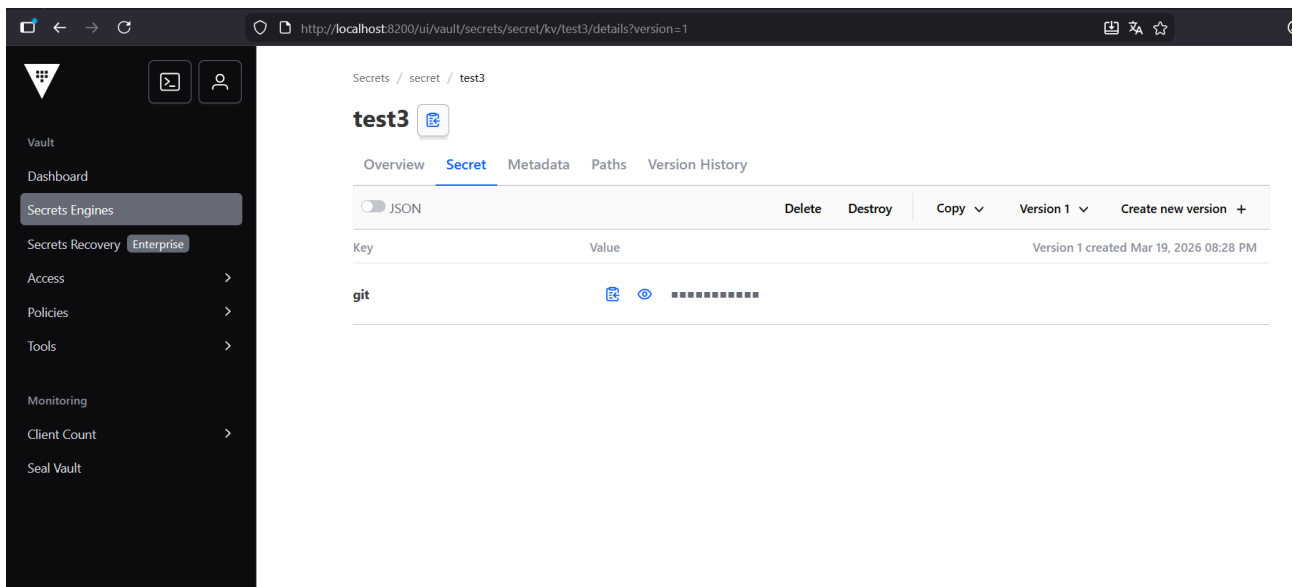


Figure 10 Interface GUI : secret test3

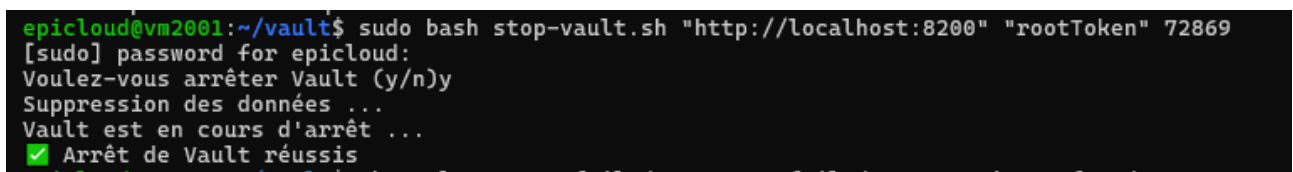


Figure 11 Arrêt de Vault

## 6. Conclusion

Ce POC démontre que Vault peut être déployé et opéré simplement, grâce au mode de développement et à un script Bash pour gérer son cycle de vie. Les deux interfaces disponibles CLI et API REST permettent d'interagir avec les secrets de manière flexible selon les besoins.

Dans le cadre du projet CIA, Vault constitue une brique de sécurité essentielle : il centralise les secrets de l'infrastructure et évite de les stocker en clair dans les scripts ou fichiers de configuration. La prochaine étape naturelle serait de passer d'un déploiement en dev mode à un déploiement en mode production, avec persistance des données et gestion du processus d'unsealing.